FRONTEND ROADMAP

TW

Tailwind CSS Notes

A utility-first CSS framework: style elements straight from your HTML with small, composable classes. Your reference for the classes you'll reach for every day.

WHAT WE COVER

Utility-first | Spacing & color | Flexbox & Grid | Responsive | Best practices

Tailwind changes how you write CSS. Instead of naming things and jumping to a separate stylesheet, you describe the look right on the element with tiny classes like `p-4` and `flex`. These notes walk through the core utilities, responsive design, and the habits that keep Tailwind code clean.

Tailwind CSS Notes

Tailwind gives you single-purpose classes you combine directly in your markup — no custom class names, no context-switching to a CSS file. Learn the vocabulary once and you can build almost any interface by composing these building blocks.

ON THIS HANDOUT

01 What is Tailwind? Utility-first CSS

Tailwind is a **utility-first** CSS framework. Rather than writing CSS rules, you apply many small pre-made classes directly on your elements — each doing one small thing.

HTML

```
<!-- Traditional CSS: name a class, then write a rule for it -->
<button class="btn-primary">Buy</button>
<style> .btn-primary { background:#4f46e5; color:#fff; padding:8px 16px; } </style>

<!-- Tailwind: describe the look right on the element -->
<button class="bg-indigo-600 text-white px-4 py-2 rounded-lg">Buy</button>
```

- Each class does **one** thing: a colour, some padding, a font size.
- You **compose** them to build any design — little to no custom CSS.
- No inventing class names, no separate stylesheet to keep in sync.

02 Getting started

The quickest way to try Tailwind is the CDN — one script tag and you can start adding classes. Perfect for learning and demos.

```HTML
<!DOCTYPE html>
<html>
<head>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body>
  <h1 class="text-3xl font-bold text-indigo-600">Hello Tailwind!</h1>
</body>
</html>
```

For real projects you'd install Tailwind via npm (`npm install tailwindcss`) so it scans your files and ships only the classes you actually use — a much smaller final CSS file. The CDN is ideal while learning.

03 Spacing — padding, margin, gap

Spacing classes follow a simple scale where each step is about 4px (`p-1` =4px, `p-2` =8px, `p-4` =16px...).

```HTML
<div class="p-4">     padding on all sides       </div>
<div class="px-6 py-2"> horizontal 6, vertical 2   </div>
<div class="mt-8">     margin-top only            </div>
<div class="flex gap-4">even space between children</div>
```

- `p` =padding, `m` =margin. Add a side: `t` top, `b` bottom, `l` left, `r` right, `x` horizontal, `y` vertical.
- `gap-4` spaces items inside a flex or grid container.

04 Colour & typography

Colours use a name plus a shade from 50 (lightest) to 900 (darkest). Text utilities control size, weight and alignment.

```HTML
<p class="text-gray-800">dark grey text</p>
<p class="bg-indigo-600 text-white">coloured background, white text</p>

<p class="text-sm">small</p>
<p class="text-2xl font-bold">big & bold</p>
<p class="text-center">centered text</p>
```

- Shades scale evenly: `gray-100` (light) → `gray-900` (dark). Same for indigo, red, green, blue...
- Sizes step up: `text-xs` , `text-sm` , `text-base` , `text-lg` , `text-xl` , `text-2xl` ...
- Weights: `font-normal` , `font-medium` , `font-semibold` , `font-bold` .

05 Borders, rounding & shadow

These three give plain boxes a finished, card-like look.

```HTML
<div class="border rounded-lg shadow-md p-4">a soft card</div>

<div class="rounded-full">a pill or circle</div>
<div class="border-2 border-indigo-500">thicker, coloured border</div>
```

- Rounding: `rounded`, `rounded-lg`, `rounded-xl`, `rounded-full`.
- Shadow depth: `shadow-sm`, `shadow-md`, `shadow-lg`.

06 Layout with Flexbox

Add `flex` to a parent, then align its children. This handles most one-dimensional layouts — rows and stacks.

```HTML
<div class="flex items-center justify-between">
  <span>Left</span>
  <span>Right</span>
</div>

<div class="flex flex-col gap-4">   <!-- stack vertically -->
  <div>Row 1</div>
  <div>Row 2</div>
</div>
```

- `flex` turns on flexbox; `flex-col` stacks items vertically.
- `items-center` aligns cross-axis; `justify-between` spreads items apart.
- `justify-center`, `items-start`, `gap-4` cover the common cases.

07 Layout with Grid

For two-dimensional layouts — equal rows and columns like galleries or dashboards — reach for grid utilities.

```HTML
<div class="grid grid-cols-3 gap-6">
  <div>1</div> <div>2</div> <div>3</div>
  <div>4</div> <div>5</div> <div>6</div>
</div>
```

- `grid-cols-3` makes 3 equal columns — change the number to change the layout.
- `gap-6` spaces cells evenly in both directions.
- Combine with `col-span-2` to make a cell wider.

08 Responsive design

Tailwind is **mobile-first**: a class with no prefix applies at every size. Add a breakpoint prefix to override it on larger screens.

```HTML
<!-- 1 column on phones, 2 on tablets, 4 on desktops -->
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6"> ... </div>

<!-- text grows with the screen -->
<h1 class="text-2xl md:text-4xl">Responsive heading</h1>
```

- Breakpoints: `sm:` 640px, `md:` 768px, `lg:` 1024px, `xl:` 1280px.
- Write the mobile version first, then layer on bigger-screen overrides.

09 States: hover, focus & more

Prefixes also target interaction states, so you can style hover and focus without any extra CSS.

```HTML
<button class="bg-indigo-600 hover:bg-indigo-700 focus:ring-2 focus:ring-indigo-300
               px-4 py-2 rounded-lg text-white transition">
  Hover or focus me
</button>
```

- Common states: `hover:` , `focus:` , `active:` , `disabled:` .
- Add `transition` for smooth changes between states.

10 Common styling patterns

A few patterns you'll reuse constantly. Copy, then tweak the utilities to taste.

Card

```HTML
<div class="max-w-sm rounded-xl bg-white p-6 shadow-md">
  <h3 class="text-lg font-semibold text-gray-800">Card title</h3>
  <p class="mt-2 text-sm text-gray-500">A short description goes here.</p>
</div>
```

Button

```HTML
<button class="rounded-lg bg-indigo-600 px-4 py-2 font-medium text-white
               hover:bg-indigo-700">
  Primary
</button>
```

Alert / badge

```html
<div class="rounded-lg bg-green-100 px-4 py-3 text-sm text-green-700">Saved!</div>
<span class="rounded-full bg-red-100 px-2 py-0.5 text-xs font-medium text-red-600">-20%</span>
```

11 Reusable components with @apply

When you keep repeating the same long class list, pull it into a single class with `@apply` inside your CSS — the readability of a named class with the power of utilities.

```css
.btn {
  @apply rounded-lg bg-indigo-600 px-4 py-2 font-medium text-white hover:bg-indigo-700;
}
```

Now `<button class="btn">Save</button>` gives you the whole styled button. Use this for things repeated many times (buttons, inputs); for one-offs, plain utilities are fine.

12 Best practices

- **Mobile-first:** style the small screen first, then add `md:` / `lg:` overrides.
- **Extract when it repeats:** copy-pasting the same 8 classes 10 times? Make a component or use `@apply`.
- **Don't fight the scale:** prefer `p-4`, `p-6` over arbitrary values — it keeps spacing consistent.
- **Group by concern:** order classes as layout → spacing → colour → type → states so they're easy to scan.
- **Use the docs:** you don't memorise every class — search `tailwindcss.com` as you go.
- **Install for production:** the npm build strips unused classes, keeping your CSS tiny.

> **QUICK RECAP**
>
> Tailwind = **small utility classes** composed in your HTML · **flex** and **grid** handle layout · prefixes like **md:** and **hover:** add responsiveness and interaction · extract repeated class lists with **@apply**. Build big things by composing small ones.